OPERATIONAL INSTRUCTION MANUAL

# School Management Software Production Readiness, Security, Quality & Optimization Playbook

## FINAL EDITION

| | |
|---|---|
| **Purpose** | Instruct an AI coding agent to inspect, test, repair, optimize and verify an **existing** school management software project, with evidence for every claim. |
| **Intended audience** | Antigravity IDE agent (any model strength) and the human owner reviewing its work. |
| **Scope** | Technology-agnostic until the repository is inspected. Feature-specific checks activate only if the feature is discovered. |
| **Structure** | Sections 0–28; execution phases numbered 0–8, each with defined gates; final execution block addressed to Antigravity. |
| **Status of this document** | Instructions only. It does not state or imply that any audit, testing or remediation has been performed on the application. |

# Table of Contents

**0. HOW TO USE THIS DOCUMENT (READ FIRST) ..... 4**
0.1 Vocabulary ..... 4
0.2 Status terms (USE ONLY THESE, EVERYWHERE) ..... 4
**1. NON-NEGOTIABLE RULES ..... 5**
R1 — The repository is the source of truth ..... 5
R2 — Preserve existing functionality ..... 5
R3 — Evidence is mandatory ..... 5
R4 — Controlled execution (per-phase loop) ..... 5
R5 — Safe execution & human approval ..... 6
R6 — Do not stop at the audit ..... 6
**2. SAFETY TRIPWIRES (STOP CONDITIONS) ..... 6**
**3. DELIVERABLE FILES (create/update; adapt names if equivalents exist—review before overwriting) ..... 7**
**4. PROGRESS LEDGER (AUDIT_PROGRESS.md) — CREATE IN PHASE 0 ..... 7**
**5. PHASE OVERVIEW (EXECUTE IN ORDER; DO NOT SKIP SILENTLY) ..... 8**
5.1 Per-phase report format (MANDATORY at end of every phase) ..... 9
**6. PHASE 0 — SAFETY & BASELINE ..... 9**
**7. PHASE 1 — DISCOVERY ..... 10**
7.1 Repository inventory (write to ARCHITECTURE_INVENTORY.md) ..... 10
7.2 Feature inventory (per feature; write to FEATURE_TEST_MATRIX.md) ..... 10
7.3 Entry-point inventory (every externally reachable surface) ..... 10
7.4 Role & permission inventory ..... 10
7.5 Critical workflow dependency maps ..... 11
7.6 Risk-based priority list ..... 11
**8. PHASE 2 — SECURITY AUDIT & AUTHORIZED ATTACK SIMULATION ..... 11**
8.1 Setup ..... 11
8.2 Security Testing Matrix ..... 12
8.3 Per-finding remediation loop (MANDATORY) ..... 14
8.4 Authorization matrix (create in SECURITY_FINDINGS.md) ..... 14
8.5 Security execution safeguards and recording (applies to every row of 8.2) ..... 14
**9. PHASE 3 — FUNCTIONAL QUALITY (CONDITIONAL MODULES) ..... 15**
9.1 Functional Test Matrix ..... 16
9.2 Edge-case checklist (test each that applies; add discovered ones) ..... 17
**10. PHASE 4 — TARGETED REMEDIATION ..... 18**
**11. PHASE 5 — PERFORMANCE & OPTIMIZATION (MEASURE FIRST) ..... 18**
11.1 Performance Measurement & Optimization Matrix ..... 19
11.2 Optimization record (per change) ..... 19
11.3 Optimization guard rules ..... 20
**12. PHASE 6 — RELIABILITY, PRIVACY, ACCESSIBILITY ..... 20**
12.1 Reliability tests (local/disposable; use fault injection, e.g., stop DB container, block provider via mock) ..... 20
12.2 Privacy & school-data protection ..... 20
12.3 Accessibility (WCAG 2.2 AA where applicable; ref: w3.org/TR/WCAG22) ..... 21

**13. AUTOMATED TEST STRATEGY (use EXISTING stack first) ..... 21**

**14. DATABASE MIGRATIONS, BACKUPS, DISASTER RECOVERY (PHASE 6/7) ..... 21**

**15. DEPLOYMENT, CONFIG & OPERATIONS (PHASE 7) ..... 22**

15.1 Antigravity MAY do autonomously (local / disposable only) ..... 22

15.2 Antigravity MUST NOT do autonomously ..... 22

15.3 Production-dependent checks (TLS/headers on the live endpoint, live health checks, live monitoring/alerting, production backup status, live config values, etc.) ..... 22

15.4 Code quality (run throughout) ..... 22

**16. TEMPLATES ..... 23**

16.1 Severity model ..... 23

16.2 Risk register (RISK_REGISTER.md) ..... 23

16.3 Defect report (copy per finding) ..... 23

16.4 Test execution report (TEST_EXECUTION_REPORT.md) ..... 24

16.5 Phase tracker ..... 24

**17. TEST DATA & ENVIRONMENT RULES ..... 24**

**18. LOW-MODEL RELIABILITY PROTOCOL (MANDATORY) ..... 24**

18.1 Decision rules (use when unsure) ..... 25

**19. RELEASE-READINESS CRITERIA (FINAL GATE) ..... 25**

**20. REFERENCES (verify availability before citing; do not invent versions) ..... 26**

**21. FINAL HANDOVER (PHASE 8) ..... 26**

**22. RECOVERY AFTER INTERRUPTION OR CONTEXT LOSS ..... 26**

**23. EXECUTION BUDGET & CONTEXT MANAGEMENT (MANDATORY) ..... 27**

**24. VERSION CONTROL, CHECKPOINT & ROLLBACK (MANDATORY) ..... 27**

**25. REAL UI & BROWSER TESTING (MANDATORY) ..... 28**

25.1 Capability discovery (Phase 0, task P0-8) ..... 28

25.2 Rules ..... 28

25.3 UI test record (one per route/workflow tested) ..... 29

**26. TEST-FAILURE HANDLING (MANDATORY; strengthens the two-strike rule) ..... 29**

**27. LARGE-SCALE AUDIT MANAGEMENT & COVERAGE TRACKING (MANDATORY) ..... 29**

**28. APPROVAL REQUESTS (TEMPLATE) ..... 30**

> **Audience:** Antigravity IDE agent (any model strength). **Target:** the EXISTING school management project in the open workspace. **Mode:** inspect → test → fix → verify → document. Do NOT rebuild. Do NOT stop at a report. **Technology:** unknown until you inspect the repository. Never assume a language, framework, database, host, auth method, or feature. **Edition:** FINAL (corrected). Adds mandatory cross-phase protocols in **Sections 23–28** (execution budget, checkpoint/rollback, real UI testing, failure handling, coverage management, approvals). These apply to EVERY phase.

---

# 0. HOW TO USE THIS DOCUMENT (READ FIRST)

1. Read this whole file before acting. Re-read the current Phase section at the start of each phase.
2. Work in **small, bounded tasks**. One task at a time. Each task gets an ID (`T-<phase>-<nnn>`).
3. Every claim needs **evidence** (command output you actually observed, file path + line, test result). No evidence = `UNVERIFIED`.
4. Persist state in `AUDIT_PROGRESS.md` (Section 4) after every milestone. If your context is lost, follow Section 22 (Recovery).
5. Conditional modules (marked **[IF EXISTS]**) activate only if you discover that feature in the repo. Record "NOT APPLICABLE – feature not found (evidence: searched X, Y)" otherwise.
6. **Read Sections 23–28 before Phase 0.** They are mandatory protocols (execution budget, checkpoint/rollback, real UI testing, failure handling, coverage tracking, approvals) that apply in every phase. If any instruction here conflicts with a stricter safety rule, the **stricter safety rule wins**.
7. A feature that was not discovered is not automatically secure. An endpoint that was not tested is not automatically functional.

## 0.1 Vocabulary

| Term | Meaning |
|---|---|
| **Confirmed defect** | Reproduced with evidence |
| **Confirmed vulnerability** | Exploit path demonstrated safely in a test environment |
| **Suspected** | Code looks risky, not yet reproduced |
| **Improvement** | No defect, but beneficial |
| **UNVERIFIED** | Could not be checked; explain how to check it |
| **Gate** | Pass/fail checkpoint at end of a phase |
| **Disposable env** | Local/throwaway DB + synthetic data that can be wiped safely |
| **Checkpoint** | A verified, recoverable record of the pre-remediation project state, made through an existing authorized, non-destructive method (or one the human approved for that exact operation) — Section 24 |
| **Batch** | A small group of related tasks worked on together, sized by complexity and risk (Section 23) |

## 0.2 Status terms (USE ONLY THESE, EVERYWHERE)

| Status | Meaning |
|---|---|
| **PASS** | Executed; observed result matched expected result; evidence recorded |
| **FAIL** | Executed; observed result differed from expected result |
| **BLOCKED** | Could not be executed (missing tool, environment, approval, dependency); reason recorded |
| **NOT RUN** | Not executed yet (or deliberately deferred); reason recorded |

| Status | Meaning |
|---|---|
| **UNVERIFIED** | Believed true/false but not proven by execution; how-to-verify recorded |
| **NOT APPLICABLE** | Feature/surface does not exist; **evidence of absence** (what you searched) recorded |

Phase **gate decisions** use PASS / FAIL / BLOCKED / PARTIAL (PARTIAL is for gate decisions only, never for individual tests).

- A test the framework **skipped**, or any unexecuted test, is NOT RUN (or BLOCKED) — **never PASS**.
- A missing tool, unavailable browser, unauthorized environment, or unavailable production access makes a check BLOCKED or UNVERIFIED — **never PASS**.
- NOT APPLICABLE requires evidence that the feature or surface genuinely does not exist. A check that cannot be done for lack of access is **not** NOT APPLICABLE.
- Feature *implementation* status in Section 7.2 (complete / partial / mocked / unverified) describes the product, not test results.

**Workflow states (not test results).** Tasks, batches and phases carry a separate **workflow state** (progress only). Only the six statuses above may be used for **individual test results**.

| Workflow state | Applies to | Meaning |
|---|---|---|
| **NOT STARTED** | task, batch, phase | Work has not begun (task still in the Pending queue). Not a test result. |
| **IN PROGRESS** | task, batch, phase | Work is under way; no outcome or gate decision yet. Not a test result. |

- **NOT STARTED** and **IN PROGRESS** are workflow states only; **never** record them as a test-result status in any test record, matrix, UI test record, `TEST_EXECUTION_REPORT.md` or `COVERAGE_TRACKER.md`. An unexecuted test is NOT RUN.
- Gate decisions (PASS / FAIL / BLOCKED / PARTIAL) exist only once a phase gate is evaluated; work still IN PROGRESS is never forced into a test-result status or gate decision.
- A finished, failed, blocked or deferred task moves to the matching ledger queue; a phase records its gate decision. Queue names are not test-result statuses.

---

# 1. NON-NEGOTIABLE RULES

## R1 — The repository is the source of truth

- Inspect before changing anything. Documentation/README is NOT proof a feature works; trace the real code path.
- Never invent files, commands, routes, tables, env vars, dependencies, results, or features.
- "Not found" ≠ "not vulnerable". "Test not run" ≠ "test passed".
- If you cannot verify something, write `UNVERIFIED` + the exact steps a human can use to verify.

## R2 — Preserve existing functionality

- Do not rebuild working modules, swap architecture, or delete working features to make tests pass.
- Do not silently change business rules, UI behavior, API contracts, or DB semantics.
- Do not overwrite user data, existing migrations, configs, or uncommitted local changes.
- No incompatible schema change without a migration + rollback plan.
- No major dependency upgrade without a demonstrated need + compatibility analysis.
- Do not disable/skip failing tests, weaken validation, or suppress security warnings to get "green".
- Do not automatically commit, stash, reset, clean, force-push, rewrite history, or discard user changes (see Section 24).
- Prefer minimal, targeted, reversible changes.

### R3 — Evidence is mandatory

Every finding needs (see template §16.3): ID, severity, location, evidence, repro steps, expected vs actual, root cause, fix, regression test, verification result, remaining uncertainty. Classify each as: (1) Confirmed defect, (2) Confirmed vulnerability, (3) Suspected, (4) Improvement, (5) Unverified. Never say "passed" because code was generated, read, or assumed.

### R4 — Controlled execution (per-phase loop)

1. Read relevant files → 2. Establish baseline → 3. Define exact scope → 4. Write/update tests → 5. Smallest safe fix → 6. Run targeted tests → 7. Run regression tests → 8. Review your diff → 9. Record evidence → 10. Proceed only if the gate passes.

- **Two-strike rule:** after two failed attempts at the same root cause, STOP, write down the evidence, form a *different* diagnosis, then try again. No random retries.
- Never mark failed/skipped checks as passed.
- Detailed failure handling: Section 26. Execution budget and context management: Section 23.

### R5 — Safe execution & human approval

All testing runs on local/disposable or explicitly authorized staging only. STOP and ask the human before:

- Any production DB write/migration, or deleting/overwriting real data
- Production deployment
- Disabling auth or any security control
- Sending real emails/SMS/notifications
- Real payments/financial transactions
- Rotating/revoking production credentials
- Load tests that could hit shared infrastructure
- Changing DNS/firewall/infra access
- Major dependency upgrades, destructive schema changes, deleting tests
- Committing, stashing, resetting, cleaning, force-pushing, rewriting history, or discarding/overwriting user changes (approval is for the exact operation only; Section 24)
- Processing real student/guardian information in any form

Never run destructive tests against production. Never print secrets in logs, reports, screenshots, or code.

#### R5.1 Scope of permission (unambiguous)

- Permission to audit and repair the local codebase does NOT imply permission to modify production systems or process real student information.
- A general instruction such as "finish everything", "fix all", or "continue without waiting" is NEVER authorization for destructive, production, external-communication, payment, credential, or infrastructure actions.
- When approval is needed: record the exact operation, the risk, the options, and why approval is needed (template in Section 28), add it to "Outstanding approvals" in `AUDIT_PROGRESS.md` , and continue independent safe work.
- Routine, safe, reversible tasks on the local/disposable environment (reading code, writing tests, targeted local fixes, running local tests) proceed autonomously.

### R6 — Do not stop at the audit

After finding defects, implement safe fixes, add regression tests, and verify. The goal is a measurably improved, tested, documented product.

## 2. SAFETY TRIPWIRES (STOP CONDITIONS)

Stop immediately and ask the human if any occurs:

- ☐ The configured DB/URL/credentials appear to point at production or shared data.
- ☐ A command would delete/truncate/drop data, force-push, rewrite git history, or discard/overwrite user changes.
- ☐ Tests would send real messages or charge real money (provider keys are live, not sandbox).
- ☐ You find real student/guardian data in the repo or test DB (do not copy it; report location only).
- ☐ A secret is found committed in the repo (report file+line, NOT the value; recommend rotation; human performs rotation).
- ☐ Working tree has uncommitted changes you did not make and your task would touch the same files.

Secrets handling: when reporting a leaked secret write `[REDACTED — type: API key, file: path:line]` only.

## 3. DELIVERABLE FILES (create/update; adapt names if equivalents exist—review before overwriting)

Place in an existing docs directory (discover it) or `docs/audit/` .

| # | File | Purpose |
|---|---|---|
| 1 | `AUDIT_PROGRESS.md` | Ledger (Section 4) — update constantly |
| 2 | `AUDIT_SUMMARY.md` | Exec summary, readiness, risks, blockers |
| 3 | `ARCHITECTURE_INVENTORY.md` | Stack, entry points, data flows, integrations |
| 4 | `FEATURE_TEST_MATRIX.md` | Features × scenarios × status |
| 5 | `SECURITY_FINDINGS.md` | Vulnerabilities, evidence, fixes, retests |
| 6 | `PERFORMANCE_REPORT.md` | Baselines, bottlenecks, before/after |
| 7 | `BUG_AND_REMEDIATION_LOG.md` | Defects, root cause, fix, regression |
| 8 | `TEST_EXECUTION_REPORT.md` | Commands, counts, environment |
| 9 | `RELEASE_READINESS_CHECKLIST.md` | Go/no-go |
| 10 | `DISASTER_RECOVERY.md` | Backup/restore, RPO/RTO |
| 11 | `KNOWN_LIMITATIONS.md` | Untested/unfixed/unverifiable |
| 12 | `RISK_REGISTER.md` | Template §16.2 |
| 13 | `COVERAGE_TRACKER.md` | Every discovered route/role/workflow/applicable security test with a status (Section 27) |

Never put passwords, tokens, keys, or real student data in any of these.

## 4. PROGRESS LEDGER ( `AUDIT_PROGRESS.md` ) — CREATE IN PHASE 0

Update after every task/batch, after every gate, and before ending any session. The ledger is the **single source of truth** for resuming; do not rely on conversation history.

```
# AUDIT PROGRESS LEDGER
- Last updated: <timestamp>
- Current phase: <0-8>   Phase workflow state: <NOT STARTED|IN PROGRESS>
- Last phase gate decision (gate decisions only): <PASS|FAIL|BLOCKED|PARTIAL>
- Current task ID: <T-x-nnn>   Task workflow state: <NOT STARTED|IN PROGRESS>
- Current batch ID: <B-x-nn>
- Environment classification: <local-disposable | staging-authorized | UNKNOWN(treated as productio
n)>
- Repository state at last verification:
  - VCS: <git | none | other>   Branch: <name>   HEAD: <commit hash>
  - Working-tree summary: <`git status --short` summary>
  - Pre-existing user changes (not mine): <file list>
  - Checkpoint: <reference/location + how to restore> | NOT CREATED (limitation: <why>)
- Baseline (from Phase 0): build=<result> lint=<result> types=<result> tests=<passed/failed/
skipped/not-run>
- Baseline failures (pre-existing): <list with IDs>
- Newly introduced failures (caused by my changes): <list or NONE>
- Last successful test run: <command, timestamp, counts>

## Completed tasks
| ID | Description | Evidence (file/command/output ref) | Date |
## Failed tasks
| ID | Description | Attempt count | Attempted diagnoses & results | Next different diagnosis |
## Blocked tasks
| ID | Description | Exact blocker | Needs human? | Independent work continuing? |
## Pending tasks (PRIORITY ORDER: P0/P1 + critical workflows first)
| ID | Description | Priority | Complexity (S/M/L) | Depends on |
## Deferred queue (cannot be done this session)
| ID | Description | Reason | Prerequisite |
## Files modified (with reason and task ID)
## Commands run and results (summary; full logs in TEST_EXECUTION_REPORT.md)
## Outstanding approvals (Section 28 format)
## Outstanding questions for human
## Recovery instructions (how to restore pre-remediation state; see Section 24)
## Next exact action
<ONE concrete instruction: which file, which command, which task ID. NOT "continue auditing".>
```

Before resuming from this ledger you MUST inspect the current working tree and verify that the recorded assumptions (branch, HEAD, modified files, environment, baseline) still hold. See Section 22.

---

## 5. PHASE OVERVIEW (EXECUTE IN ORDER; DO NOT SKIP SILENTLY)

| Phase | Name | Gate (must be true to proceed) |
|---|---|---|
| 0 | Safety & baseline | No destructive ops; baseline recorded; user work protected; checkpoint verified (or limitation documented); capabilities discovered |
| 1 | Discovery | Scope & critical assets documented |
| 2 | Security audit | High-risk findings identified, reproduced where feasible, prioritized |
| 3 | Functional quality | Critical workflow status evidence-backed |
| 4 | Targeted remediation | No P0 uncontained; no regressions |
| 5 | Performance & resources | Optimizations evidence-backed; behavior intact |
| 6 | Reliability, privacy, a11y | Unresolved risks explicitly documented |
| 7 | Full regression & release verification | Verdict supported by actual evidence |
| 8 | Final handover | Reports done; truthful verdict |

If a phase is blocked: document the blocker, continue only with independent safe tasks. Phases 2–3 may interleave with 4 (fix P0/P1 as soon as confirmed), but record each in the ledger.

### 5.1 Per-phase report format (MANDATORY at end of every phase)

```
**Phase name:**
**Scope inspected:**
**Files and components inspected:**
**Baseline:**
**Findings:**
**Changes made:**
**Tests added or updated:**
**Commands executed:**
**Actual results:** (passed / failed / skipped / not run / blocked / unverified — real counts)
**Regression results:**
**Security implications:**
**Remaining risks:**

**Coverage status (from `COVERAGE_TRACKER.md`):**

**Rollback/recovery procedure for changes made this phase:**

**Approvals outstanding:**
**Gate decision:** PASS / FAIL / BLOCKED / PARTIAL
**Reason for gate decision:**
```

If a command could not run, say why. Never infer success from an exit code you did not observe.

---

## 6. PHASE 0 — SAFETY & BASELINE

**Tasks**

☐ P0-1 Confirm repository root (`pwd`, list top-level files). Note the VCS (likely git).

☐ P0-2 Inspect version control (Section 24): confirm VCS, current branch, `git status`, `git log -5 --oneline`, and read the diff of pre-existing changes. List uncommitted changes — these are the **user's work; protect them**. **Never automatically commit, stash, reset, clean, switch branches, or discard changes.** Use an **existing, explicitly authorized, non-destructive** checkpoint method if one is available. If creating a checkpoint would need a prohibited operation, file an approval request (Section 28) for that exact operation and **continue independent safe inspection and testing**. Record whether a checkpoint exists and how recovery would work; if none is possible, record the limitation and restrict later work to small, reviewable, safely reversible changes.

☐ P0-3 Identify environment: where is the DB, what do `.env*` files point to (host names only, never values of secrets)? Classify: local-disposable / staging / production. If unsure → treat as production and ask.

☐ P0-4 Discover scripts/tools: read package manifests/build files (`package.json`, `pyproject.toml`, `requirements*.txt`, `pom.xml`, `build.gradle`, `composer.json`, `Gemfile`, `go.mod`, `Cargo.toml`, `*.csproj`, `Makefile`, `Dockerfile`, `docker-compose*`, CI files) — whichever exist.

☐ P0-5 Capture BASELINE (run only what exists; record exact command + real output summary + counts):

- dependency install (reproducible? lockfile present?)
- build / production build
- lint, formatter check, type check
- unit / integration / e2e tests
- Record **pre-existing failures** separately: they are baseline, not caused by you.

☐ P0-6 Create `AUDIT_PROGRESS.md`, `TEST_EXECUTION_REPORT.md` (baseline section).

☐ P0-7 Prepare safe test environment: disposable DB, synthetic seed data, sandbox/mocked providers, email/SMS sinks. If none exists, document the minimum setup needed and create it locally (no production access).

☐ P0-8 **Capability discovery (Section 25):** can the app be started? Is browser automation, console/network inspection, viewport testing, screenshot capture, keyboard/accessibility checking available? Record each as AVAILABLE / UNAVAILABLE. Mark browser-only checks BLOCKED/UNVERIFIED where unavailable.

☐ P0-9 Create `COVERAGE_TRACKER.md` skeleton (Section 27) and the first task queue with IDs, complexity, and dependencies (Section 23).

**Gate 0:** no destructive operation performed; baseline documented (including pre-existing failures); user work protected; environment classified; checkpoint verified OR limitation documented; capabilities recorded; ledger created. Report Phase 0 baseline to the human, then proceed to Phase 1 (no need to wait unless a tripwire or approval boundary was reached).

---

# 7. PHASE 1 — DISCOVERY

### 7.1 Repository inventory (write to `ARCHITECTURE_INVENTORY.md`)

Discover and record with file-path evidence:

- Languages, frameworks, package managers, versions
- Folder structure & architecture (monolith/modular/services; frontend/backend split)
- APIs (REST/GraphQL/RPC/server actions), background jobs, schedulers, webhooks, integrations
- DB engine, schema, relationships, migrations, indexes, constraints, seed scripts
- Auth: method, sessions/tokens, roles, account recovery, MFA
- Files: uploads, storage provider, exports/imports, generated documents (PDF, CSV, XLSX)
- Existing tests, linters, type checking, CI/CD
- Env var **names** and purpose (never values); config validation
- Deployment, logging, monitoring, caching, backups
- Dependencies, versions, deprecated/vulnerable packages (run the project's own audit tool, e.g. `npm audit`, `pip-audit`, `composer audit`, `bundle audit`, `cargo audit`, `govulncheck`, `dotnet list package --vulnerable`, OWASP Dependency-Check — only those that exist/are installed)
- TODO/FIXME/HACK, mock data, hardcoded demo values, stubbed functions, placeholder buttons, fake success responses (use text search: `TODO|FIXME|HACK|mock|stub|dummy|lorem|placeholder|not implemented`)

### 7.2 Feature inventory (per feature; write to `FEATURE_TEST_MATRIX.md`)

```
| Feature | Frontend components | Backend routes/actions | DB entities | Permissions | Integrations
| Dependencies | Existing tests | Known failures | Status: complete / partial / mocked / unverified
|
```

### 7.3 Entry-point inventory (every externally reachable surface)

```
| Type (page/API/server action/webhook/file handler/admin/job) | Path | Method | Auth required? |
Roles allowed | Handler file | Notes |
```

### 7.4 Role & permission inventory

Find roles from code (enums, DB tables, policy files, middleware). Never assume "admin/teacher/student/parent" exist — record what actually exists.

### 7.5 Critical workflow dependency maps

For each workflow that **actually exists**, draw a text chain, e.g.:

`Login → session issued → role resolved → authorization check → DB query`

Candidates [IF EXISTS]: admission, attendance, fees/payments, exams/results, notices, uploads/downloads, imports/exports, notifications.

### 7.6 Risk-based priority list

Rank assets: (1) money, (2) credentials/sessions/admin functions, (3) student/guardian PII & minors' data, (4) grades/results, (5) files, (6) everything else.

**Gate 1:** inventories written; critical assets and entry points listed; missing/unknown areas marked UNVERIFIED.

---

## 8. PHASE 2 — SECURITY AUDIT & AUTHORIZED ATTACK SIMULATION

**Scope limits:** only this repo, local/authorized env, synthetic accounts, harmless payloads, bounded request counts. No third-party targets, no DoS, no real-data exfiltration, no data destruction.
**Standards:** OWASP Top 10, OWASP API Security Top 10, OWASP ASVS, OWASP Cheat Sheet Series (owasp.org). Record which published version/date you used (check the official site if accessible; otherwise write "version not verified"). Do **not** claim "OWASP compliant" or "penetration tested".

### 8.1 Setup

- Create ≥2 synthetic users per discovered role, plus ≥2 synthetic tenants/schools [IF multi-tenant], each with distinct records ("A" owns record A1; "B" owns B1).
- Prepare a harmless marker payload set (e.g., `<b>xss-marker-123</b>`, `'"--`, `../marker`, `{{7*7}}`, long string, Unicode). They must not do anything destructive.

## 8.2 Security Testing Matrix

| ID | Attack category | Preconditions | Safe test procedure | Expected secure result | Regression test |
|---|---|---|---|---|---|
| S-01 | **IDOR / BOLA** | Users A,B with own records | Logged in as A, request B's record by changing ID in UI/API/export/download URL (GET, PUT, DELETE) | 403/404, no B data in body, no modification | Automated test: A→B's ID on every object endpoint returns denied; B's record unchanged |
| S-02 | **BFLA / vertical escalation** | Low-privilege role user | Call admin/higher-role endpoints directly (no UI); try admin pages by URL | 401/403; no side effects | Test per role × privileged endpoint |
| S-03 | **Horizontal escalation** | Two same-role users | Read/modify peer's data (e.g., other teacher's class, other parent's child) | Denied | Same-role cross-owner test |
| S-04 | **Cross-tenant isolation** [IF multi-school] | Two tenants | Cross-tenant IDs in search, reports, exports, files, jobs, cache | Zero cross-tenant data | Tenant-isolation suite incl. lists/aggregates |
| S-05 | **Mass assignment** | Create/update endpoints | Send extra fields (`role`, `isAdmin`, `schoolId`, `ownerId`, `paid`, `balance`) | Protected fields ignored/rejected | Test that protected fields are unchanged after request |
| S-06 | **Client-trusted values** | Fees/marks endpoints | Send altered price/total/ID/role/status | Server recomputes/validates | Test server-side recomputation |
| S-07 | **SQL/NoSQL injection** | Any DB-backed input (search, filters, sort, IDs, login) | Harmless payloads (`'`, `" OR 1=1 --`-style *read-only* probes) on local DB; static review for string-built queries | Parameterized; no error leak; no changed results | Test with payloads asserting normal handling |
| S-08 | **Command/ template/ expression injection** | Code that shells out, renders templates, evals | Static search (`exec`, `system`, `spawn`, `eval`, `Function(`, `shell=True`, template-from-string); harmless probes (`{{7*7}}`) | No execution; input treated as data | Unit test with probe string |
| S-09 | **Stored/reflected/ DOM XSS** | Any user text rendered (names, notices, comments, filenames, rich text) | Store marker payload; view as another role; check raw HTML/DOM; search for unsafe sinks (`dangerouslySetInnerHTML`, `v-html`, `innerHTML`, `\|safe`, `raw`, `html_safe`) | Payload displayed as text, no script/handler injected | Test render output is escaped/sanitized |
| S-10 | **Open redirect / URL schemes** | Redirect/`next`/`returnUrl` params, user links | Supply external URL / `javascript:` | Allowlisted/relative only | Test redirect validator |
| S-11 | **Clickjacking / headers / CSP** | Running app | Inspect response headers: CSP, `X-Frame-Options`/`frame-ancestors`, `X-Content-Type-Options`, HSTS (prod), Referrer-Policy, Permissions-Policy | Present & sensible; no wildcard where avoidable | Header test in integration suite |

| ID | Attack category | Preconditions | Safe test procedure | Expected secure result | Regression test |
|---|---|---|---|---|---|
| S-12 | **CSRF** | Cookie-based sessions | Cross-origin form/fetch without token on state-changing endpoints | Rejected (token and/or SameSite + origin check) | Test missing/invalid token rejected; GET never mutates |
| S-13 | **CORS** | API | Send `Origin: https://untrusted.example` (test value) incl. preflight | No wildcard with credentials; untrusted origin not reflected | Test CORS allowlist |
| S-14 | **Session security** | Auth working | Inspect cookie flags (`HttpOnly`, `Secure` in prod, `SameSite`), session fixation (ID changes at login), logout invalidation, post-password-change invalidation, token in URL, tokens in localStorage | Secure flags; server-side invalidation works | Tests for logout/password-change invalidation |
| S-15 | **Brute force / rate limiting** | Login/reset/OTP endpoints | **Bounded** local attempts (e.g., ≤30) | Throttle/lockout after threshold; uniform responses | Rate-limit test with low threshold in test config |
| S-16 | **Account enumeration** | Login/register/reset | Compare responses/timing for existing vs non-existing users | Indistinguishable messages/status | Test equal responses |
| S-17 | **Password reset / token security** | Reset flow | Reuse token, expired token, token for other user, predictable token, token in logs/URL referrer | Single-use, short-lived, high entropy, bound to user | Reuse/expiry tests |
| S-18 | **Password storage / crypto** | Code review | Hashing algorithm (should be bcrypt/scrypt/argon2/PBKDF2 w/ proper params), no reversible/plain/MD5/SHA1-only; secure RNG for tokens | Strong adaptive hash; CSPRNG | Unit test hash format (do not print hashes) |
| S-19 | **File upload** [IF EXISTS] | Upload endpoint | Harmless synthetic files: wrong extension/MIME, `.html/.svg` with script marker, `../` filename, oversize, empty, corrupt, double extension, zip with `../` entry (synthetic) | Rejected or neutralized; random server-side names; stored outside web root or served with safe headers (`Content-Disposition`, `nosniff`); size limits | Upload validation test suite |
| S-20 | **File download authorization** | Private files | Fetch another user's file by URL/ID/guessed path; unauthenticated fetch | Denied; no directory listing/public bucket | Authorization test on download route |
| S-21 | **Path traversal** | Any file path built from input | `../` variants on download/template/export params | Rejected/normalized | Test traversal strings |
| S-22 | **SSRF** [IF server fetches user URLs] | URL import/webhook-target/avatar-by-URL | Use **test doubles**; submit loopback, private ranges, link-local, redirect-to-private. Do NOT contact real metadata endpoints | Blocked by allowlist + resolved-IP validation + redirect re-validation | Unit tests of URL validator with fake resolver |
| S-23 | **Webhook security** [IF EXISTS] | Payment/other webhooks | Missing/invalid signature, replayed event, out-of-order event, duplicate event | Signature verified (constant-time, raw body); idempotent; replay window | Tests: bad signature rejected; duplicate has no second effect |
| S-24 | **Business-logic abuse** | Workflows | Invalid state transitions (e.g., unpublish→publish skipping approval, pay a cancelled invoice, edit locked results), negative/zero/huge amounts, replay requests | Server enforces state machine & ranges | State-transition tests |
| S-25 | **Concurrency / race** | Money, counters, unique numbers | Fire N (small, e.g., 10) parallel identical requests on local env | One effect only; unique constraint/lock/idempotency holds | Parallel-request test |

| ID | Attack category | Preconditions | Safe test procedure | Expected secure result | Regression test |
|---|---|---|---|---|---|
| S-26 | **Excessive data exposure** | API responses | Inspect JSON for password hashes, tokens, internal IDs, other users' fields, over-broad lists | Only fields needed by that role | Response-schema allowlis test |
| S-27 | **Resource consumption** | List/search/export endpoints | `limit=100000`, deep pagination, huge payload (bounded locally) | Enforced max page size/body size/ timeouts | Test caps |
| S-28 | **Error/log leakage** | Error paths | Trigger errors; inspect responses & logs | No stack traces/SQL/secrets/PII to users; logs free of passwords/tokens | Test error response shape |
| S-29 | **Secrets in repo/ history** | Whole repo | Search for key patterns; check `.gitignore`, committed `.env`; scan history with an *installed* scanner if available | No secrets committed (else REDACTED report + human rotates) | Add secret-scan to CI if absent (recommend) |
| S-30 | **Dependency / supply chain** | Lockfiles, CI | Run ecosystem audit tool; check lockfile consistency, install scripts, unpinned deps, CI permissions & secret exposure, container base image | Known-exploitable vulns resolved; reproducible installs | Audit step in CI (recommend) |
| S-31 | **Authz in cache/ background jobs** | Caches, queues, reports | Cache key lacks user/tenant; job runs without scoping | Keys include tenant/user/role; jobs scoped | Test cache isolation |
| S-32 | **Spreadsheet formula injection** | CSV/XLSX export | Store value starting `=`, `+`, `-`, `@` in a text field; export | Cell neutralized (e.g., prefixed `'`) | Export test |
| S-33 | **Audit logging** | Sensitive actions | Perform grade change, fee edit, role change, export | Logged with actor/time/target, no secrets | Test log entry exists |

### 8.3 Per-finding remediation loop (MANDATORY)

1. Reproduce safely → record evidence. 2. Write a failing regression test. 3. Implement the targeted fix. 4. Confirm exploit path now blocked. 5. Confirm legitimate behavior still works. 6. Search for the **same pattern** in sibling routes/files and fix/log them. 7. Record residual risk. 8. Update ledger + `SECURITY_FINDINGS.md`.

### 8.4 Authorization matrix (create in `SECURITY_FINDINGS.md`)

Rows = discovered **resources/actions** (read, create, update, delete, export, admin). Columns = discovered **roles** + "unauthenticated" + "other tenant". Cell = expected ALLOW/DENY (derived from code/business rules; ask the human if the intended rule is unclear), and later: observed result + test ID. Test **both** the UI path and the **direct API** path.

### 8.5 Security execution safeguards and recording (applies to every row of 8.2)

- Run only in a local, disposable, or explicitly authorized staging environment, with synthetic users/records, harmless proof-of-concept payloads, bounded request counts and bounded time. Stop any test showing unexpected resource consumption or affecting shared infrastructure.
- No destructive testing against production; no real-data exfiltration; no uncontrolled denial of service; no attacks on unrelated third-party systems; no secrets in findings or logs.
- For every applicable test record: preconditions, procedure, expected secure behavior, actual outcome, evidence, remediation, regression test, and status (Section 0.2).

- **Both directions:** each negative test (unauthorized action denied) needs a matching positive test (legitimate authorized behavior still works).
- Prioritize order: object-level authorization → privilege escalation → cross-school isolation → session security → injection → XSS → CSRF → file access → SSRF (if applicable) → webhook validation → rate limiting → business-logic abuse → sensitive-data exposure.
- Use representative tests to diagnose a pattern, then **systematically sweep all affected endpoints/code** (Section 27). Do not claim comprehensive coverage from a sample.
- Do not claim a complete penetration test, certification, or compliance status because static analysis or scanners passed.
- Use staging only with the human's explicit authorization, recorded in the ledger. Never use production.
- **Never weaken or remove a security control to make a test pass.** If a test needs a relaxed setting (e.g., a low rate-limit threshold), use a test-only configuration in the disposable environment, record it, and confirm production configuration is unchanged.
- No real payments, no real emails/SMS/notifications, no infrastructure changes, no secrets in logs or reports.

**Gate 2:** every row of the security matrix and every authorization-matrix cell has an explicit status (PASS / FAIL / BLOCKED / NOT RUN / UNVERIFIED / NOT APPLICABLE-with-evidence) in `COVERAGE_TRACKER.md`; all high-risk findings are recorded, reproduced where feasible, and prioritized.

---

## 9. PHASE 3 — FUNCTIONAL QUALITY (CONDITIONAL MODULES)

Activate each module only if the feature exists. For each scenario, record status: PASS / FAIL / BLOCKED / NOT RUN / UNVERIFIED / NOT APPLICABLE (Section 0.2), with evidence. **Verify persisted state (DB/API), not just the UI message.**

## 9.1 Functional Test Matrix

| Module | Scenario | Expected result | Verification method |
|---|---|---|---|
| **Auth** | Register/login/logout; wrong password; malformed input; expired/invalid/ tampered token | Clear error; no partial access; session ends on logout | API + browser; check back-buttor after logout; DB session state |
| Auth | Password reset incl. reuse/expired token; verify email/phone | Single use; expires | Test + mailbox sink |
| Auth | Disabled/suspended/deleted user logs in or uses existing session | Denied immediately | Disable then reuse token |
| Auth | Duplicate account (case/whitespace/Unicode variants of email) | One account only | DB unique check |
| Auth | Concurrent sessions; role change takes effect | Old permissions not retained | Change role, reuse session |
| **RBAC** | For every role: view/create/update/delete/export/admin | Matches authorization matrix | Direct API + UI; manual URL entry |
| **Students** | Create/edit/view/archive; unique admission number; required/optional fields; long/Unicode/leading-zero IDs; duplicates & double submit; concurrent edit; transfer/class change; guardian links | Valid data saved once; invalid rejected with clear message; history preserved | UI + API + DB row counts |
| Students | Search/sort/filter/paginate incl. empty result & records changing mid-pagination | Stable, correct, bounded | Compare with DB query |
| **Staff/Classes/ Subjects/ Timetable** | CRUD; duplicate assignment; reference to nonexistent record; timetable conflicts (teacher/room/class double-booked); archive keeps history; academic-year rollover | Conflicts rejected; FK integrity | API + DB |
| **Attendance** | Mark/edit/view; duplicate entry; future/invalid date; holiday/non-working day; school timezone & midnight boundary; bulk partial failure; summaries/percent math; unauthorized edit of past records | One record per student/date/session; totals consistent; partial failure doesn't corrupt | Recompute summary independently; compare to report |
| **Fees/Payments** | Fee structure → invoice; discount/concession/late fee/rounding; partial payment & balance; receipts; duplicate submit; webhook duplicate/out-of-order; failed/ pending/cancelled/refund; decimal precision; other student's invoice access; unauthorized refund/edit | Money math exact (no float drift); invoice marked paid only after **authoritative** provider confirmation persisted; idempotent | **Sandbox/mock only.** Recomput totals independently; reconcile invoice ↔ payment ↔ receipt |
| **Exams/Results/ Report cards** | Mark range validation; grade boundaries & rounding; absent/exempt; recalculation after edit; unpublished results hidden from students/parents; export = DB; promotion keeps history | Calculations match spec; no leakage of unpublished data | Independent recalculation; role-based access test |
| **Notices/Events/ Notifications** | Draft vs published visibility; audience targeting; edit/archive; duplicate notifications; failed delivery & retry; attachments/links | Only intended audience sees; no duplicates; failures logged | Role view test; email/SMS sink |
| **Import/Export/ Reports** | Valid CSV/XLSX/PDF; invalid ext/MIME; empty/oversize/malformed; Unicode/ regional names; duplicate rows; partial failure → rollback; export vs DB equality; large report; download authorization/expiry; temp-file cleanup; audit trail | Atomic import (all-or-nothing or clear partial report); exports match DB | Compare counts/values; inspect temp dir |
| **UI integrity** | Click every control on every route (route inventory): dead buttons, silent form failures, wrong links, fake success, stuck spinners, empty/error states, confirm/ cancel, refresh mid-operation, double-click, network offline/slow, long text, big tables, modals, console errors, failed network requests | Each control works or is honestly disabled; errors visible & actionable | Browser run (only claim if you actually ran it) + API logs |

| Module | Scenario | Expected result | Verification method |
|---|---|---|---|
| **Data integrity** | Constraints/FKs; orphan rows; partial transactions; concurrent writes; timezone/ date handling; NULL/empty values; soft delete; audit history; cache invalidation; dashboard = detail = export totals | Consistent everywhere | SQL consistency queries (read-only) |

**UI rows require real execution.** A UI scenario may be marked PASS only if the page/workflow was actually opened and exercised (Section 25). Static code inspection, a successful build, or an API test is NOT proof that the UI works; without browser capability, mark the UI check BLOCKED or UNVERIFIED.

## 9.2 Edge-case checklist (test each that applies; add discovered ones)

- ☐ Double-click submit
- ☐ Two admins edit same record
- ☐ Stale tab after logout/role change
- ☐ Cached old permissions
- ☐ Duplicate/out-of-order/delayed webhooks
- ☐ DST change (if timezone has it)
- ☐ Academic-year rollover
- ☐ Leap day/month-end
- ☐ Unicode/non-Latin/RTL names
- ☐ Leading-zero IDs/roll numbers (not coerced to numbers)
- ☐ Very large school dataset
- ☐ Empty search results
- ☐ Pagination while data changes
- ☐ Deleted user with historical records
- ☐ Import fails halfway
- ☐ Export interrupted
- ☐ Orphaned uploaded files
- ☐ Inconsistent dashboard aggregates
- ☐ Concurrent fee updates
- ☐ Duplicate admission
- ☐ UI vs API behavior mismatch
- ☐ Stale cache after mutation
- ☐ Clock skew/expired tokens
- ☐ Retry loops
- ☐ Partial third-party outage

- ☐ Missing/malformed env vars at startup
- ☐ Tab suspended & reconnect
- ☐ Unexpected HTTP status/content type
- ☐ Resource cleanup after errors

**Gate 3:** every critical workflow has an explicit Section 0.2 status with evidence (no critical workflow left NOT RUN/UNVERIFIED without a documented reason); each FAIL has a defect report; regression tests added for reproduced defects.

---

## 10. PHASE 4 — TARGETED REMEDIATION

Order: **P0 → P1 (security & data integrity first) → P2 → P3**. Cosmetic work last.

Rules:

- One fix per change-set; keep diffs small and reviewable/reversible.
- Write the failing test first when feasible, then fix, then rerun.
- Read the full existing implementation before editing. Name exact files before editing.
- No whole-file rewrites when a targeted edit works. No unrelated refactors.
- Schema changes: write migration + validation + rollback note; test on disposable DB; ask human before anything touching production.
- After each fix run: targeted test → module regression → full relevant suite. Record counts.
- Use the **Defect Report** (§16.3) for each. Mark resolved only when the original reproduction fails safely **and** legitimate behavior passes.
- **Before substantial remediation:** record in the ledger whether a verified checkpoint exists and how recovery would work (Section 24). If none exists, record the limitation and restrict changes to small, reviewable, safely reversible ones. A missing checkpoint never stops safe inspection or testing.
- **After every remediation batch:** review the diff → run relevant tests → compare with the recorded baseline → identify newly introduced failures → record rollback procedure. Revert only your own isolated changes, and only when appropriate and safe; never overwrite unrelated user changes.
- Source-code rollback and database rollback are different: any DB change needs its own recovery plan and disposable-DB validation.
- Failing tests: follow Section 26 exactly.

**Gate 4:** no P0 uncontained; every fix verified; no new failures vs. baseline (or new failures documented and fixed).

---

## 11. PHASE 5 — PERFORMANCE & OPTIMIZATION (MEASURE FIRST)

Rule: **no baseline → no optimization claim.** Record environment, dataset size, method, results. Use production builds. Never trade correctness, security, accessibility, or integrity for speed. Don't claim field performance from one local run.

## 11.1 Performance Measurement & Optimization Matrix

| Area | What to measure / inspect | Method (use tools present in the repo or standard browser/DB tooling) | Common fixes (only if evidence supports) | Acceptance |
|---|---|---|---|---|
| Frontend bundle | JS/CSS size, route splitting, unused deps | Production build output/bundle analyzer if available | Route-level code splitting, remove dead deps, lazy-load heavy components | Measured size drop, no behavior change |
| Core Web Vitals | LCP, INP, CLS | Lighthouse/DevTools on production build (lab data); real-user data only if exists | Image sizing/compression, font loading, avoid layout shift, reduce main-thread work | Targets (good @75th pct, field data): LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 — lab results labeled as lab |
| Rendering | Re-renders, huge tables, listeners/subscriptions cleanup, memory growth | Profiler, heap snapshots | Pagination/virtualization (when justified), memoization where measured, cleanup effects | Measured improvement |
| Network | Payload size, compression, duplicate/waterfall requests, polling, caching headers, redirects, third-party scripts | Network panel / `curl -w` timings | Select fields, pagination, gzip/brotli, HTTP cache headers for static assets, dedupe requests | Fewer/lighter requests |
| API latency | p50/p95 per key endpoint, response size | Bounded local load (small N, explicit stop condition) with synthetic data volume stated | See DB rows | Before/after table |
| Database | N+1, unbounded queries, missing/redundant indexes, composite index order, slow aggregates, lock waits, transaction length, pool size | `EXPLAIN` / `EXPLAIN ANALYZE` (engine-specific), slow-query logs, ORM query logging | Eager loading/batching, targeted index (measure read gain vs write cost), pagination, rewrite query | Plan improved; migration + rollback documented |
| Caching | Candidate computed/static data | Define: cache key (incl. tenant/user/role), TTL/invalidation, owner, staleness tolerance, failure behavior | HTTP/static caching; server cache for non-sensitive/shared-safe data | Invalidation tests pass; **no cross-user/tenant leakage**; no public caching of private data |
| Concurrency/ scale | Concurrent users, connection saturation, locks, queue backlog, session store (stateless for scaling?), file storage scaling | Controlled load test with limits (**ask human** if shared infra) | Pool tuning, queues, backpressure, rate limits | Measured; state assumptions |
| Memory/ resources | Leaks, unclosed handles/connections, unbounded collections/queues/logs, timers | Long-run bounded test, resource counters | Close/cleanup, cap sizes | Stable over run |
| Background jobs | Duration, retries, idempotency, retry storms | Job logs | Backoff, batching, idempotency keys | Bounded retries |

## 11.2 Optimization record (per change)

Baseline → bottleneck evidence → change → after-measurement → test conditions → functional regression result → security impact → maintenance trade-off. no measurable gain: do not call it an improvement; consider reverting.

### 11.3 Optimization guard rules

- Capture the baseline (environment/versions, dataset size, conditions, method, initial result) BEFORE changing anything performance-related.
- Do NOT introduce caching, indexes, architectural changes, concurrency, or new dependencies without a demonstrated need (bottleneck evidence).
- After each optimization, verify it did not introduce: cross-user or cross-school cache leakage; stale authorization decisions; incorrect totals/financial calculations; race conditions; DB inconsistencies; accessibility or functional regressions.
- If the improvement could not be measured, label it **"unverified improvement"**, not "proven optimization".

**Gate 5:** each retained optimization has evidence; full regression still passes.

---

## 12. PHASE 6 — RELIABILITY, PRIVACY, ACCESSIBILITY

### 12.1 Reliability tests (local/disposable; use fault injection, e.g., stop DB container, block provider via mock)

- ☐ DB unavailable
- ☐ external timeout
- ☐ provider rate-limit (429)
- ☐ expired credentials
- ☐ invalid config at boot
- ☐ partial transaction mid-failure
- ☐ interrupted upload
- ☐ failed background job
- ☐ duplicate retry
- ☐ malformed/empty/unexpected provider response
- ☐ refresh mid-operation
- ☐ restart during long task
- ☐ graceful shutdown
- ☐ retry limit + backoff
- ☐ idempotent operations
- ☐ transaction boundaries

Expected: operation either fully succeeds or fully fails; user sees an actionable message with **no** stack traces, SQL, tokens, or secrets; no false "success".

### 12.2 Privacy & school-data protection

Treat student/guardian data as sensitive (minors). Inspect:

- ☐ Collection of unnecessary personal data
- ☐ PII (names, DOB, addresses, guardian contacts, medical/disability **if present**, finance, grades, documents) exposure in API responses, URLs, logs, analytics, error reports
- ☐ Least-privilege visibility per role/school relationship
- ☐ Retention, archival, deletion/export procedures
- ☐ Backup retention
- ☐ Access logging for sensitive records
- ☐ Third-party sharing (analytics, email, SMS, storage)
- ☐ Consent/notice mechanisms where the product has them

Do **not** claim legal compliance (e.g., any specific privacy law). Instead list "Legal/contractual review needed: <jurisdiction/deployment unknown>" for the human. Use synthetic data only.

### 12.3 Accessibility (WCAG 2.2 AA where applicable; ref: w3.org/TR/WCAG22)

Semantic HTML • keyboard-only navigation • visible focus • dialog focus trap/return • accessible names/labels • form error association • color contrast & non-color status cues • zoom/text resize • responsive (narrow mobile, tablet, laptop, wide) • touch targets • table semantics • accessible authentication • reduced motion • alt text • live-region announcements. Look for: horizontal overflow, clipped dialogs, overlapping content, unreadable tables, unusable mobile forms, missing empty states, confusing confirmations. Automated tools (if installed) don't replace manual keyboard checks; state which you actually did. Don't redesign the UI—fix specific defects.

**Gate 6:** remaining risks explicitly documented in `KNOWN_LIMITATIONS.md`.

---

## 13. AUTOMATED TEST STRATEGY (use EXISTING stack first)

| Layer | Cover |
|---|---|
| Unit | validation, business rules, permission logic, money/grade/attendance calculations, date/timezone helpers, error handling |
| Integration | DB persistence, API contracts, authN/authZ, transactions/rollback, storage, external adapters (mocked), webhook verification |
| End-to-end | critical journeys per role, form submission, permissions, error recovery, tenant isolation |
| Regression | every confirmed defect and vulnerability |
| Contract/compat | API schemas, migrations apply cleanly from empty and from prior schema, integrations |
| Property/fuzz (bounded) | boundary values, malformed inputs, with size/time limits |

**Test quality checks:** flaky tests, order dependence, shared mutable fixtures, missing cleanup, assertions that cannot fail, mocking the behavior under test, tests that don't hit the intended path, trivially inflated coverage. Run a new test against the *unfixed* code to prove it fails first when feasible. Coverage % is informational only.

---

## 14. DATABASE MIGRATIONS, BACKUPS, DISASTER RECOVERY (PHASE 6/7)

☐ Migrations apply cleanly on an empty disposable DB; and from the previous schema version
☐ rollback/roll-forward plan per migration
☐ constraints & indexes match intent
☐ Backup mechanism exists? automated? access-controlled? encrypted (where appropriate)? retention?
☐ **Restore rehearsal** into a disposable environment with synthetic data; run integrity checks (row counts, FK checks, app smoke test); include uploaded files + metadata
☐ Document RPO/RTO as **measured or stated by human**, never invented

A backup that was never restored has **UNVERIFIED** restorability. Never delete backups or alter production recovery settings without approval. Write to `DISASTER_RECOVERY.md`.

---

# 15. DEPLOYMENT, CONFIG & OPERATIONS (PHASE 7)

Goal: build a release checklist from what **actually exists**, separating checks that can be done **locally and autonomously** from checks that need **production access or human approval**.

## 15.1 Antigravity MAY do autonomously (local / disposable only)

☐ Inspect deployment configuration (Dockerfiles, compose files, CI/CD definitions, deploy scripts, manifests).

☐ Validate environment-variable **names** and configuration structure; check env validation at boot. **Never expose secret values.**

☐ Run an available production build **locally** and record the result.

☐ Check migration scripts and deployment manifests (apply to a disposable DB only).

☐ Validate health/readiness-check implementation locally (start the app locally and call the endpoint).

☐ Review CI/CD configuration, pipeline permissions, and secret handling.

☐ Test rollback procedures in a **disposable** environment.

☐ Check: dev vs prod config separation • debug mode off in prod config • secrets injected not committed • logging, error monitoring, metrics, alerting configuration • reproducible installs (lockfile, pinned versions) • artifact integrity • scheduled jobs • timezone/time-sync assumptions.

☐ Identify missing operational controls and document the evidence.

## 15.2 Antigravity MUST NOT do autonomously

- Deploy to production.
- Change live production configuration.
- Run migrations against production data.
- Change DNS, firewall rules, infrastructure access, or live credentials.
- Execute production writes or destructive operations.
- Claim a live-environment check passed when it was not performed.

## 15.3 Production-dependent checks (TLS/headers on the live endpoint, live health checks, live monitoring/alerting, production backup status, live config values, etc.)

- If actual production verification is unavailable or unauthorized, mark the check **BLOCKED**, **UNVERIFIED**, or **NOT RUN**, record the **precise reason**, and write the **steps an authorized human must follow** to verify it (into `RELEASE_READINESS_CHECKLIST.md`).
- **Never** label a production-dependent check NOT APPLICABLE merely because the agent lacks access. NOT APPLICABLE requires evidence the surface does not exist.
- TLS/security-header checks run only against an endpoint the human has explicitly authorized.
- The final release verdict (Section 19) must reflect these limitations honestly.

Produce `RELEASE_READINESS_CHECKLIST.md` listing every check with a Section 0.2 status.

## 15.4 Code quality (run throughout)

Run existing formatter/linter/type-check/static analysis. Look for: unsafe casts, missing input schemas, duplicated business logic across layers, inconsistent validation, dead code, circular deps, excessive complexity, inconsistent error handling, excessive/PII logging. Fix confirmed defects and high-value issues only; no rewrite. Never silence a diagnostic without a written justification.

---

# 16. TEMPLATES

## 16.1 Severity model

| Level | Definition | Release impact |
|---|---|---|
| **P0 Critical** | Confirmed severe exposure/unauthorized access to sensitive data, major financial-integrity failure, immediate threat | **Blocks release** until contained + independently verified |
| **P1 High** | Serious exploitable defect, significant corruption risk, critical workflow broken | **Blocks release** unless human grants documented, time-bounded exception |
| **P2 Medium** | Material reliability/privacy/authz/performance/ accessibility/correctness issue, limited impact | Fix or schedule |
| **P3 Low** | Minor defect / low-risk / maintainability | Backlog |
| **P4 Info** | Observation / optional enhancement | Optional |

Severity = evidence + exploitability + impact + exposure − compensating controls (not category name). Never hide serious findings behind an average score.

## 16.2 Risk register (`RISK_REGISTER.md`)

| Risk ID | Description | Asset | Likelihood (L/M/H) | Impact (L/M/H) | Severity | Evidence | Mitigation/ Owner | Status (open/ mitigated/ accepted) | Accepted by / expiry (if exception) |
|---|---|---|---|---|---|---|---|---|---|

## 16.3 Defect report (copy per finding)

```
### <ID e.g., F-001> — <title>
- Type: Confirmed defect | Confirmed vulnerability | Suspected | Improvement | Unverified
- Severity/Priority: P0–P4
- Affected: <file:line / route / endpoint / DB object>
- Evidence: <what you observed; command/output excerpt (no secrets)>
- Steps to reproduce:
  1.
- Expected behavior:
- Actual behavior:
- Root cause:
- Proposed remediation:
- Regression test: <test file/name>
- Fix applied: <summary + files>
- Attempt log: <attempt 1: diagnosis → change → result; attempt
2: ...> (after 2 failed attempts on the same root cause: STOP and re-diagnose, Section 26)
- Rollback procedure for this fix:
- Verification: <commands + real results; exploit now blocked? legit path OK?>
- Similar patterns checked: <list>
- Remaining uncertainty:
- Defect lifecycle status: OPEN | FIXED-VERIFIED | FIXED-UNVERIFIED | BLOCKED | ACCEPTED-RISK
(individual test results use Section 0.2 statuses)
```

### 16.4 Test execution report ( `TEST_EXECUTION_REPORT.md` )

```
## Run <n> — <phase/task ID>
- Environment: OS, runtime versions, DB (disposable/staging), seed data version
- Command: `<exact command>`
- Date/time:
- Result: passed=<n> failed=<n> skipped=<n> not-run=<n> blocked=<n> unverified=<n>
- Failures (name → cause → finding ID):
- Skipped tests and why (each skip needs a justification):
- Compared against baseline run: <run ID>
- Baseline (pre-existing) failures vs newly introduced failures:
- Notes / UNVERIFIED items:
```

### 16.5 Phase tracker

| Phase | Workflow state | Gate decision | Gate evidence | Date |
|---|---|---|---|---|
| 0 | | | | |
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |
| 8 | | | | |

---

## 17. TEST DATA & ENVIRONMENT RULES

- Synthetic students/teachers/guardians/finance records only; deterministic seeds; never copy production data.
- Separate unit / integration / e2e data; tests must be re-runnable; clean up only the records you created.
- Email/SMS to sinks or mocks; payments to provider sandbox/mocks; no production credentials.
- Use separate test tenants to verify isolation.
- Record seed + cleanup procedure in `TEST_EXECUTION_REPORT.md` .
- No sensitive data in fixtures or screenshots.
- If no safe test environment exists, creating one is the first remediation task.

---

## 18. LOW-MODEL RELIABILITY PROTOCOL (MANDATORY)

1. Reread the current phase and ledger at the start of each phase.
2. Keep `AUDIT_PROGRESS.md` current (completed/failed/blocked/pending).
3. Unique task IDs; **one bounded task at a time**.
4. Name exact files before editing; read the existing code first.
5. No speculative edits across unrelated files; no whole-file replacement when a targeted edit is safer.
6. Never assume a command exists—inspect scripts/docs/installed tools. Prefer existing tools; don't install new ones when an equivalent exists.
7. Never assume an env var, API, schema, or route exists.

8. Never invent test output or screenshots; never claim a browser test you did not run.
9. Don't mark resolved until the original reproduction fails safely and legitimate behavior passes.
10. Don't remove failing tests unless proven invalid, documented, and human-approved.
11. Keep changes small enough to review/revert.
12. When uncertain: **gather evidence, don't guess**. If a decision needs human judgment, present choices + risks and wait.
13. "Not found" ≠ "not vulnerable"; "not run" ≠ "passed"; don't claim exhaustive testing.
14. Before ending any session: persist progress + the next exact action.
15. "Test thoroughly" is never an instruction by itself. For every test record: what you ran (exact command/action), what you expected, what you observed, where the evidence is, and the Section 0.2 status.
16. State uncertainty instead of guessing. This playbook does not promise zero errors or complete security coverage; report limits honestly in `KNOWN_LIMITATIONS.md`.

### 18.1 Decision rules (use when unsure)

| Situation | Do this |
|---|---|
| Test fails after your change | Read the failure; revert only your change if cause unclear; diagnose; two-strike rule |
| Test was failing before you started | Record as baseline failure; fix only if in scope/safe; never hide |
| Fix needs a schema change | Write migration + rollback; test on disposable DB; ask human before shared/prod |
| Intended business rule unclear | Ask the human; don't guess; mark task BLOCKED and continue other work |
| Dependency has a vulnerability | Check if reachable/exploitable; prefer patch/minor update; run full tests; major upgrade → ask human |
| Command could affect shared/prod data | STOP, ask |
| Checkpoint would need a prohibited operation (commit/stash/reset/clean/history change) | File an approval request for that exact operation (Section 28); continue independent safe work |
| Tool not installed | Use an alternative already present; else document as UNVERIFIED (don't install heavy tools silently) |
| Finding is in many places | Fix one with a test, then sweep the pattern, log each location |

---

## 19. RELEASE-READINESS CRITERIA (FINAL GATE)

Mark **READY** only if ALL are true:

☐ Local production build and local deployment-configuration checks pass (Section 15.1); every critical **production-dependent** check (Section 15.3) has actually been performed by an authorized party and PASSED — otherwise READY is not allowed
☐ Critical business workflows pass defined tests
☐ No unresolved/uncontained P0
☐ P1s resolved, or have a documented, authorized, time-bounded exception from the human
☐ Critical authorization boundaries tested (UI **and** API) incl. tenant isolation [IF applicable]
☐ No known critical data-integrity defect
☐ Migrations have reviewed rollback/recovery strategy
☐ Backups/restore verified to the level claimed (state exactly what was verified)
☐ Important automated tests pass; no failures hidden by skips/suppressed diagnostics

- ☐ Material performance regressions ruled out or explicitly accepted
- ☐ Privacy & operational limitations documented
- ☐ All UNVERIFIED areas disclosed in `KNOWN_LIMITATIONS.md`

**Verdict definitions**

- **READY** — all critical gates above pass **with recorded evidence**.
- **CONDITIONALLY READY** — only when every remaining condition and risk is explicitly documented, appropriately authorized by the human, assigned to an owner, and time-bounded where applicable, and **no release-blocking risk is concealed**. Typical example: all local gates pass but production-dependent checks are BLOCKED/UNVERIFIED pending authorized human verification (list the exact steps).
- **NOT READY** — required when any critical blocker remains unresolved, OR evidence is insufficient to justify release (including "the application has not been tested enough to support a conclusion").

Never claim production readiness if testing was insufficient to support it. A successful build or static scan alone never establishes production readiness. A high category score never overrides a critical security, authorization, financial-integrity, or data-integrity failure. The verdict report must separately list work that is passed, failed, skipped, not run, blocked, and unverified. Include a category scorecard (Security, Functionality, Data integrity, Performance, Reliability, Privacy, Accessibility, Tests, Operations: 0–5 with justification), but **critical blockers override any score**.

---

## 20. REFERENCES (verify availability before citing; do not invent versions)

- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- OWASP API Security Top 10 — https://owasp.org/www-project-api-security/
- OWASP ASVS — https://owasp.org/www-project-application-security-verification-standard/
- OWASP Cheat Sheet Series — https://cheatsheetseries.owasp.org/
- WCAG 2.2 — https://www.w3.org/TR/WCAG22/
- Core Web Vitals (web.dev) — https://web.dev/articles/vitals
- Official docs of the discovered framework, language, and database (EXPLAIN, migrations, backup/restore) — locate after Phase 1.

If you cannot open a reference, write "reference not verified" instead of paraphrasing from memory as fact.

---

## 21. FINAL HANDOVER (PHASE 8)

Produce all Section 3 files. `AUDIT_SUMMARY.md` must include: stack summary, what was tested, confirmed findings by severity with status, fixes verified (with commands + real counts), performance deltas (measured), unresolved blockers, UNVERIFIED list, exact commands to reproduce the final test/build run, and the verdict (READY / CONDITIONALLY READY / NOT READY). The final report MUST separately list: tests passed; tests failed; tests skipped; tests not run; tests blocked; findings fixed **and retested**; findings fixed **but not independently verified**; remaining confirmed defects; suspected issues; UNVERIFIED areas; human approvals still required; external prerequisites still incomplete. Include the final `COVERAGE_TRACKER.md` summary.

---

## 22. RECOVERY AFTER INTERRUPTION OR CONTEXT LOSS

1. Open `AUDIT_PROGRESS.md` → read "Current phase", "Current task ID", "Next exact action".

2. Re-read this playbook's section for that phase.
3. **Inspect the current working tree** (`git status`, `git diff --stat`, current branch, HEAD) and compare with the ledger's "Repository state at last verification". **Verify every recorded assumption still holds** (environment classification, baseline, files modified, checkpoint status as recorded). Investigate any difference BEFORE editing; never overwrite changes you did not record.
4. Re-run the last recorded test command to re-establish current state.
5. Check the last 2 entries of `TEST_EXECUTION_REPORT.md` and open items in `BUG_AND_REMEDIATION_LOG.md`.
6. Resume at the recorded next action. Do **not** redo completed tasks or guess what was done; if the ledger is ambiguous, verify with evidence first.
7. Add a ledger note: `Resumed from task <task ID> at <timestamp>`.
8. If another session/agent may have been active, confirm no overlapping edits (Section 23, rule 13) before touching shared files.

---

## 23. EXECUTION BUDGET & CONTEXT MANAGEMENT (MANDATORY)

This playbook is large. Weaker models can exhaust context, make too many edits at once, or lose unfinished work. Therefore:

1. Divide all work into **small, independently verifiable tasks**; give each a unique ID (`T-<phase>-<nnn>`); group related tasks into batches (`B-<phase>-<nn>`).
2. Work on **one bounded task, or one small related batch, at a time**.
3. Before starting a batch record in the ledger: task IDs, **estimated complexity (S/M/L)**, and **dependencies**.
4. **Set batch size by the complexity and risk of the actual changes.** There is no universal maximum number of files or tasks; large/risky changes → smaller batches.
5. **Priority order:** confirmed P0/P1 risks → critical functionality (auth, authorization, money, student data, grades) → everything else.
6. Never begin multiple large refactors at the same time.
7. **Persist progress after every meaningful milestone** (task done, test run, fix applied, blocker found).
8. Maintain the queues: pending, completed, failed, blocked, deferred (see ledger).
9. Record exact files changed and exact tests run for every task.
10. Resume from the persistent ledger, **not** from conversation history.
11. Re-read the relevant playbook section before resuming a phase.
12. **Never declare completion merely because the context or session is ending.**
13. If the environment supports multiple sessions/agents: do not let separate agents edit overlapping files simultaneously without coordination (record file ownership in the ledger).
14. **When context, time, or tool limits approach:** stop starting new tasks → safely finish or checkpoint the current task → update the ledger → write the **Next exact action** → stop.

---

## 24. VERSION CONTROL, CHECKPOINT & ROLLBACK (MANDATORY)

**Before substantial remediation (Phase 4 and any earlier fix):**

1. Inspect the working tree and current branch (if Git is available).
2. Identify pre-existing user changes; record the file list in the ledger.

3. Inspect the diff of any file you will touch before touching it.
4. **Choose a checkpoint method in this order:** (a) an **existing, explicitly authorized, non-destructive** method that is already available (e.g., a commit/branch/tag/backup the human already made, or a procedure the human has documented as authorized); (b) otherwise, file an approval request (Section 28) for the **exact operation** you propose; (c) if neither is possible, go to "If a safe checkpoint cannot be created" below.
5. **Never create a checkpoint by overwriting user files or by changing Git history.**
6. If you or the human created a checkpoint, **verify it exists** (e.g., the commit/branch/archive is listed and readable) and record where it is.
7. Document in the ledger ("Recovery instructions") how to restore the pre-remediation state.
8. Keep secrets and real student data out of backups and reports unless unavoidable and approved.

**Prohibited without explicit human approval of the exact operation:** committing, stashing, resetting, cleaning, force-pushing, rewriting history, discarding or overwriting user changes. A general approval ("go ahead") does not cover them.

**If a safe checkpoint cannot be created:** state the limitation in the ledger and `KNOWN_LIMITATIONS.md`, and restrict work to changes that are small, reviewable, and safely reversible by hand. **Do not stop** independent, safe inspection and testing merely because no checkpoint exists.

**For every remediation batch:** review the diff → run relevant tests → compare with baseline → identify newly introduced failures → revert only your own isolated changes when rollback is appropriate and safe → never overwrite unrelated user changes → record the rollback/recovery procedure.

**Database changes:** distinguish source-code rollback from database rollback. Every migration needs a recovery plan and validation on a **disposable** database. Never run migrations on shared/production data without approval.

**Backup verification wording:** a backup is "verified" only for the properties actually checked (existence, readability, integrity). A full restore drill (Section 14) is a separate verification activity; never describe a backup as restorable until restored.

---

## 25. REAL UI & BROWSER TESTING (MANDATORY)

Static inspection is not execution. Distinguish them explicitly.

### 25.1 Capability discovery (Phase 0, task P0-8)

Determine and record AVAILABLE / UNAVAILABLE for: starting the application locally; browser automation (use the project's existing framework where practical); browser console inspection; network-request inspection; responsive viewport testing; screenshot capture; keyboard-navigation and accessibility checking.

### 25.2 Rules

- If browser automation is available: test discovered routes and critical workflows with the project's existing framework where practical.
- If unavailable: continue with other meaningful tests, but mark browser-only checks **BLOCKED** or **UNVERIFIED** (with how a human can verify).
- **Never claim** a page was opened, a button clicked, a screenshot captured, or a workflow passed unless that action actually occurred.
- A successful build, static code reading, or an API test is **not** proof that the full UI works.

### 25.3 UI test record (one per route/workflow tested)

```
### UI-<nnn> — <route or workflow>
- Account role used:
- Preconditions and test data (synthetic):
- Interaction actually performed:
- Expected result:
- Observed result:
- Console errors / failed network requests observed:
- Evidence (screenshot path / log excerpt; no secrets or real data):
- Viewport(s) tested:
- Status: PASS | FAIL | BLOCKED | NOT RUN | UNVERIFIED | NOT APPLICABLE
```

---

## 26. TEST-FAILURE HANDLING (MANDATORY; strengthens the two-strike rule)

When a test fails:

1. Capture the actual error output and the reproduction steps.
2. Determine whether the failure **existed in the original baseline**.
3. Determine whether the **test, environment, configuration, or application** is responsible.
4. Identify the most likely root cause **using evidence**.
5. Avoid unrelated code changes.
6. Apply **one targeted correction**.
7. Run the targeted test **and** relevant regression tests.
8. Record the outcome (Attempt log in the defect report).

**After two unsuccessful attempts on the same root cause:** stop repeating similar fixes → reassess the diagnosis → collect more evidence → document whether a genuinely different approach is justified. If the root cause remains uncertain → mark the task **BLOCKED** or **UNVERIFIED** and continue only with independent, safe tasks.

**NEVER:**

- Delete a failing test to make the suite pass.
- Weaken assertions without documented justification.
- Disable authorization or validation to bypass a failure.
- Suppress errors without documented justification.
- Change unrelated modules to hide a regression.
- Mark skipped or unexecuted tests as passing.

---

## 27. LARGE-SCALE AUDIT MANAGEMENT & COVERAGE TRACKING (MANDATORY)

1. **Inventory the full scope first** (Phase 1): routes, roles, workflows, data entities, security test applicability.
2. **Group related work into risk-based batches**; prioritize sensitive data, authorization, financial integrity, and critical school workflows.
3. **Track coverage against the full inventory** in `COVERAGE_TRACKER.md`.
4. Use representative tests to diagnose patterns, then **systematically cover** the affected endpoints and related code.
5. Never claim comprehensive coverage from a small sample.

6. Use bounded test data and request counts; stop tests showing unexpected resource consumption or touching shared infrastructure.
7. Record **why** any test is skipped, blocked, or deferred.
8. Keep a separate **deferred queue** (in the ledger) for work that cannot be finished in the current session.

`COVERAGE_TRACKER.md` format (every discovered item must appear; use only Section 0.2 statuses):

| Item type | Item (route / role / workflow / security-test ID) | Role(s) tested | Test ID(s) | Status | Evidence / reason |
|---|---|---|---|---|---|
| route | | | | | |
| role × resource (authz matrix cell) | | | | | |
| workflow | | | | | |
| security test (S-nn) | | | | | |
| UI test (UI-nnn) | | | | | |

Every discovered route, every relevant role, every critical workflow, and every applicable security test MUST have an explicit status.

---

## 28. APPROVAL REQUESTS (TEMPLATE)

Use whenever an action crosses an R5 boundary. Add to "Outstanding approvals" in the ledger, present it to the human, then **continue independent safe work**.

```
### APPROVAL-<nnn>
- Exact operation requested:
- Why it is needed (task/finding ID):
- Risk if performed / risk if not performed:
- Options (including safest alternative):
- Reversibility and recovery plan:
- Environment affected: <local | staging | production>
- Status: PENDING | APPROVED (by whom, when, scope/expiry) | DENIED
```

Approval applies only to the exact operation and scope stated. It does not carry over to other operations.

---

---

# ▶▶▶ EXECUTE NOW — INSTRUCTIONS TO ANTIGRAVITY ◀◀◀

**You are Antigravity. Follow these steps in order. Do not skip steps. Report only what you actually observed.**

1. **Read this ENTIRE playbook (Sections 0–28) before acting.**
2. **Inspect the actual repository.** Do not assume the stack, features, or roles; discover them. Keep feature-specific checks conditional on what you find.
3. **Identify pre-existing user changes** (working tree, branch, diff). Never commit, stash, reset, clean, force-push, rewrite history, or discard anything without my approval of that exact operation (Section 24). Use an existing, explicitly authorized, non-destructive checkpoint method if available; otherwise request approval for the exact operation, or record the limitation and keep changes small, reviewable, and reversible. A missing checkpoint never stops safe inspection or testing.

4. **Classify the environment** (local-disposable / staging-authorized / unknown = treated as production) before running any risky command.
5. **Discover available tools and capabilities** (build, test, lint, app start, browser automation, console/network inspection, screenshots) per Section 25. Mark unavailable ones; do not pretend.
6. **Capture the actual baseline and record pre-existing failures** (Phase 0, Section 6) using only commands that exist in the project. Do not invent commands.
7. **Create each required deliverable in the discovered documentation directory or docs/audit/.** Before creating or updating a file, inspect whether an equivalent already exists and read its contents. Reuse and update an existing equivalent when appropriate, preserving its useful content and unrelated user changes. Create a new file only when no appropriate equivalent exists. Never blindly overwrite, truncate, delete, or replace an existing file. Review the proposed diff and verify that changes are limited to the audit deliverables and do not destroy user work. These deliverables include `AUDIT_PROGRESS.md` (Section 4), `COVERAGE_TRACKER.md` (Section 27), and the report files in Section 3; maintain them throughout.
8. **Execute Phase 0 first.** Report the real baseline and your Phase 0 gate decision (Section 5.1 format). Then continue to Phase 1 autonomously unless a tripwire (Section 2) or approval boundary (R5/R5.1) requires a pause.
9. **Proceed in small, independently verifiable batches** (Section 23) through Phases 1 → 8 in order.
10. **Give every task a unique ID and record the exact files you modified** (ledger).
11. **Run tests that exist in the project.** Apply **targeted fixes only to confirmed defects**, following Sections 8.3, 10, and 26.
12. **Retest the defect and the relevant legitimate workflows**; add regression tests (negative: unauthorized denied; positive: authorized still works).
13. **Inspect your diff and detect newly introduced regressions** by comparing against the recorded baseline.
14. **Record evidence and update the ledger after every milestone.**
15. **After any interruption or context loss, resume only from the ledger** (Section 22), after verifying the working tree and recorded assumptions. Never declare completion because a session is ending.
16. **Ask for approval only** at actual approval boundaries (R5/R5.1) or safety tripwires (Section 2), using the Section 28 format. "Finish everything" is never authorization for destructive, production, external-communication, payment, credential, or infrastructure actions. You must **never** deploy to production, change live configuration, or run production migrations without my explicit approval of that exact operation.
17. **Continue independent safe tasks while blocked tasks are documented** with exact reasons.
18. **State uncertainty instead of guessing.** Never fabricate evidence; never claim a UI action, screenshot, or browser test you did not perform; never mark a failed, skipped, blocked, or not-run check as PASS; never claim a live-environment check passed when it was not performed.
19. **Finish with a truthful release verdict** (READY / CONDITIONALLY READY / NOT READY, Section 19), the separated results list required by Section 21, and the exact remaining blockers, UNVERIFIED areas, human approvals still required, and external prerequisites.

**Begin with Phase 0 now.**